

Лекция 3. Основы языка Kotlin

- Типы данных, циклы, условия.
- Менеджеры компоновки, адаптеры.
- Программное создание кастомизированного интерфейса.
- RecyclerView.
- Коллекции в kotlin, структуры данных.

Kotlin — это статически типизированный язык программирования, созданный для повышения производительности разработчиков и минимизации распространенных ошибок. Его современный синтаксис сочетает лаконичность с мощными функциями, что делает его идеальным выбором для Android-разработки, серверных приложений и даже веб-фронтенда. Kotlin стал одним из ключевых языков для разработки Android-приложений благодаря трём важным преимуществам: совместимости с Java, безопасности типов и минимизации шаблонного кода.

Kotlin создан так, чтобы без проблем взаимодействовать с Java — языком, на котором изначально писались все Android-приложения. Это означает:

- Использование существующих библиотек: Все Java-библиотеки и фреймворки (например, Android SDK) доступны в Kotlin без изменений. Разработчик может вызвать Java-класс в Kotlin или наоборот.
- Постепенная миграция: Если проект написан на Java, его можно переводить на Kotlin поэтапно. Например, новые модули писать на Kotlin, а старые оставить на Java.
- Общий runtime: Kotlin компилируется в байт-код для Java Virtual Machine (JVM), что гарантирует совместимость на уровне платформы.

Язык Kotlin фокусируется на безопасности типов и сокращении boilerplate-кода. Boilerplate-код — это повторяющиеся шаблоны, которые не несут смысловой нагрузки, но требуют времени на написание. Kotlin

минимизирует его за счёт короткого синтаксиса, лямбда-выражений, делегированных свойств и Extension-функций.

Каждая переменная в Kotlin имеет тип данных, который определяет, какие значения она может хранить и какие операции с ней возможны. Тип переменной указывается один раз при её создании и не может быть изменён. Это называется статической типизацией, и она помогает избежать ошибок, связанных с неожиданным изменением типа данных.

Типы данных в Kotlin

Категория	Тип	Описание	Пример объявления	Размер (бит)	Диапазон Особенности	Примечания
Числовые (целые)	Byte	Целое число со знаком.	val a: Byte = 100	8	от -128 до 127	Используется для экономии памяти (например, в массивах).
	Short	Целое число со знаком.	val b: Short = 30_000	16	от -32768 до 32767	Редко используется, обычно для совместимости с Java-библиотеками.
	Int	Целое число со знаком (по умолчанию).	val c: Int = 2_147_483_647	32	от -2 ³¹ до 2 ³¹ - 1	Наиболее распространенный тип для целочисленных операций.
	Long	Целое число со знаком.	val d: Long = 9_223_372_036_854_775_807L	64	от -2 ⁶³ до 2 ⁶³ - 1	Суффикс L обязателен для явного указания.
Числовые (дробные)	Float	Число с плавающей точкой одинарной точности.	val e: Float = 3.14f	32	~6-7 десятичных знаков	Суффикс f обязателен.
	Double	Число с плавающей точкой двойной точности (по умолчанию).	val f: Double = 3.1415926535	64	~15-16 десятичных знаков	Используется для точных вычислений.
Символьный	Char	Символ Unicode.	val g: Char = 'A'	16	Любой символ Unicode (например, '\u0041' для 'A')	Не может быть напрямую преобразован в число (используйте .code).
Логический	Boolean	Логическое значение (true/false).	val h: Boolean = true	8 (JVM)	true или false	Используется в условиях и циклах.
Строковый	String	Неизменяемая последовательность символов.	val i: String = "Hello"	Зависит от длины	Любые символы в двойных кавычках. Тройные кавычки для raw-строк: ""Многост	Поддерживает шаблоны: "Значение: \$x".

					рочный текст"".	
Массивы	Array<T>	Фиксированный массив элементов типа T.	val j: Array<Int> = arrayOf(1, 2, 3)	Зависит от типа	Индексируется от 0 до size-1.	Для примитивов используйте специализированные массивы: IntArray, CharArray.
Коллекции	List<T>	Неизменяемый список.	val k: List<String> = listOf("a", "b")	-	Доступ только для чтения.	Изменяемая версия: MutableList<T>.
	Set<T>	Неизменяемое множество уникальных элементов.	val l: Set<Int> = setOf(1, 2)	-	Гарантирует уникальность элементов.	Изменяемая версия: MutableSet<T>.
	Map<K, V>	Неизменяемый словарь пар "ключ-значение".	val m: Map<String, Int> = mapOf("a" to 1)	-	Ключи уникальны.	Изменяемая версия: MutableMap<K, V>.
Nullable-типы	T?	Переменная может содержать null.	val n: String? = null	-	Любой тип можно сделать nullable, добавив ?.	Для безопасного доступа используйте ?. или оператор Элвиса (?:).
Специальные типы	Any	Супертип для всех типов (аналог Object в Java).	val o: Any = 42	-	Может содержать значение любого типа.	Для проверки типа используйте is или as.
	Unit	Аналог void в Java. Используется для функций без возвращаемого значения.	fun printMessage(): Unit { ... }	-	Явно возвращает Unit, но указывать его необязательно.	

Nullable-типы (T?) в Kotlin — это типы данных, которые могут содержать либо значение указанного типа T, либо null. Они позволяют явно указать, что переменная может быть неинициализированной или отсутствовать, что помогает предотвратить ошибки `NullPointerException` на этапе компиляции.

Добавьте ? к типу, чтобы сделать его nullable:

```
var name: String? = null // Допускает null
var age: Int? = 42      // Может быть null или числом
```

Компилятор Kotlin проверяет, что вы обрабатываете возможный null перед использованием nullable-переменной. Это исключает случайные `NullPointerException`.

Операторы для работы с Nullable-типами:

- Безопасный вызов (?.) выполняет действие, только если значение не null:

```
val length: Int? = name?.length // Если name == null, вернет null
```

- Elvis-оператор (?:) возвращает значение по умолчанию, если переменная null:

```
val validName = name ?: "Гость" // Если name == null, вернет "Гость"
```

- Утверждение не-null (!) принудительно преобразует nullable-тип в non-null:

```
val riskyLength = name!!.length // Выбросит NPE, если name == null
```

- Проверка на null (if):

```
if (name != null) {  
    println(name.length) // Компилятор автоматически приведет к non-null  
}
```

Примеры

1. Объявление nullable-переменных:

```
var email: String? = "user@example.com" // Может быть строкой или null  
var count: Int? = null // Изначально null
```

2. Безопасный вызов метода:

```
val upperEmail = email?.uppercase()  
// Если email == null, upperEmail тоже будет null
```

3. Работа с коллекциями:

```
val list: List<Int?> = listOf(1, null, 3) // Список может содержать null  
val sum = list.filterNotNull().sum() // Отфильтрует null и суммирует: 4
```

Kotlin разработан так, чтобы сочетать высокую производительность с надежностью кода, используя подходы, которые минимизируют накладные расходы и предотвращают распространённые ошибки. В основе его эффективности лежит автоматическая оптимизация примитивных типов: например, значения Int или Double, хотя и выглядят как объекты, внутри преобразуются в соответствующие примитивы Java (int, double) на уровне байт-кода, что устраняет затраты на создание объектов и ускоряет операции. Для работы с массивами примитивных данных Kotlin предоставляет специализированные типы вроде IntArray или ByteArray, которые хранят значения напрямую, без упаковки в объекты, что особенно важно при обработке больших объёмов информации. Параллельно язык строго разделяет типы, допускающие null, и те, которые не могут быть пустыми, что исключает случайные NullPointerException и заставляет разработчика

явно обрабатывать потенциальные пустые значения через безопасные вызовы или проверки. Ещё одной сильной стороной Kotlin является поддержка функций как полноценных типов данных, позволяющая передавать их как аргументы, возвращать из других функций или хранить в переменных, что упрощает реализацию логики, связанной с коллекциями или асинхронными операциями. Вместе с этим язык вводит понятие перечислений (enum class), которые обеспечивают типобезопасный способ работы с ограниченными наборами значений, предлагая гибкость для добавления свойств и методов к каждому элементу перечисления. Все эти механизмы работают вместе, обеспечивая баланс между скоростью выполнения, читаемостью кода и защитой от ошибок.

Условия

Условия — это конструкции в программировании, которые позволяют выполнять разный код в зависимости от выполнения определённого логического выражения. Они лежат в основе принятия решений в приложениях: например, проверка прав доступа, выбор действий на основе данных пользователя или обработка ошибок.

В Kotlin есть две основные конструкции для работы с условиями:

if-else — базовая логика «если это, то сделать то».

when — расширенная конструкция, заменяющая множественные if-else и поддерживающая сложные проверки.

Конструкция if-else проверяет условие и выполняет блок кода, если оно истинно (true), и другой блок — если условие ложно (false).

Синтаксис:

```
if (условие) {  
    // Код, если условие истинно  
} else {  
    // Код, если условие ложно  
}
```

Пример:

```
val age = 18  
if (age >= 18) {
```

```
println("Вы совершеннолетний")
} else {
    println("Вы несовершеннолетний")
}
```

Если `age >= 18` — выполняется первый блок. Иначе — второй.

В Kotlin `if` может возвращать значение, что позволяет использовать его в присваиваниях: `val result = if (a > b) "a больше" else "b больше или равно"`

Это заменяет отсутствие тернарного оператора в Kotlin (в отличие от `condition ? a : b` в Java).

Конструкция `when` — это мощная альтернатива множественным `if-else`, особенно когда нужно проверить несколько вариантов. Она работает как расширенный `switch` из других языков, но с большей гибкостью.

Синтаксис:

```
when (значение) {
    вариант1 -> { /* код */ }
    вариант2 -> { /* код */ }
    else -> { /* код по умолчанию */ }
}
```

Примеры:

- Проверка точного совпадения.

```
val day = 3
when (day) {
    1 -> println("Понедельник")
    2 -> println("Вторник")
    3 -> println("Среда")
    else -> println("Неизвестный день")
}
```

Выведет: "Среда".

- Проверка диапазонов (`in`).

```
val score = 85
when (score) {
    in 0..59 -> println("Неудовлетворительно")
    in 60..79 -> println("Удовлетворительно")
    in 80..100 -> println("Отлично")
    else -> println("Недопустимый балл")
}
```

Выведет: "Отлично".

- Проверка типов (is).

```
fun checkType(value: Any) {  
    when (value) {  
        is Int -> println("Это число: $value")  
        is String -> println("Это строка: $value")  
        else -> println("Неизвестный тип")  
    }  
}
```

Вызов `checkType(42)` выведет: "Это число: 42"

- Комбинирование условий.

```
val x = 5  
when {  
    x < 0 -> println("x отрицательное")  
    x == 0 -> println("x равно нулю")  
    else -> println("x положительное")  
}
```

Выведет: "x положительное".

Логические операторы

Для создания сложных условий используются логические операторы:

`&&` (И) — оба условия должны быть истинными.

`||` (ИЛИ) — хотя бы одно условие должно быть истинным.

`!` (НЕ) — инвертирует логическое значение.

Пример:

```
val isStudent = true  
val hasTicket = false  
  
if (isStudent && hasTicket) {  
    println("Доступ разрешён")  
} else {  
    println("Доступ запрещён")  
}
```

Выведет: "Доступ запрещён", так как `hasTicket` ложен.

Циклы

Циклы в Kotlin — это конструкции, которые позволяют выполнять блок кода несколько раз, пока выполняется определённое условие. Они являются основой для обработки повторяющихся задач, таких как перебор элементов коллекции, выполнение операций до достижения нужного результата или обработка данных в заданном диапазоне. Kotlin предоставляет несколько типов циклов, каждый из которых подходит для конкретных сценариев.

1. Цикл for

Цикл for используется для перебора элементов, таких как элементы списка, массива, диапазона или любой другой коллекции, которая поддерживает итерацию. Он особенно удобен, когда известно количество итераций заранее.

Синтаксис:

```
for (элемент in коллекция) {  
    // Блок кода, выполняемый для каждого элемента  
}
```

Примеры.

- Перебор диапазона чисел:

```
for (i in 1..5) {  
    println("Итерация $i")  
}
```

Этот код выведет числа от 1 до 5, так как диапазон 1..5 включает обе границы.

- Перебор элементов списка:

```
val fruits = listOf("Яблоко", "Банан", "Апельсин")  
for (fruit in fruits) {  
    println("Фрукт: $fruit")  
}
```

Каждый элемент списка fruits будет выведен в консоль.

- Перебор с индексами:

```
val fruits = listOf("Яблоко", "Банан", "Апельсин")  
for ((index, fruit) in fruits.withIndex()) {  
    println("Индекс $index: $fruit")  
}
```


Метод `withIndex()` позволяет получить и индекс, и значение элемента.

2. Цикл `while`

Цикл `while` выполняет блок кода до тех пор, пока условие истинно . Он используется, когда количество итераций заранее неизвестно и зависит от динамического условия.

Синтаксис:

```
while (условие) {  
    // Блок кода, выполняемый, пока условие истинно  
}
```

Пример:

```
var count = 0  
while (count < 5) {  
    println("Счёт: $count")  
    count++  
}
```

Этот код выведет числа от 0 до 4, так как цикл завершится, когда `count` станет равным 5.

Особенности:

- Условие проверяется до каждой итерации .
- Если условие ложно с самого начала, цикл не выполнится ни разу.

3. Цикл `do-while`

Цикл `do-while` похож на `while`, но с ключевым отличием: тело цикла выполняется хотя бы один раз, даже если условие сразу ложно.

Синтаксис:

```
do {  
    // Блок кода, выполняемый минимум один раз  
} while (условие)
```

Пример:

```
var result = 0  
do {  
    result += 1
```

```
println("Результат: $result")
} while (result < 5)
```

Этот код увеличит result на 1 и выведет его значение, даже если result изначально был больше 5.

Особенности:

- Условие проверяется после выполнения тела цикла .
- Используется, когда нужно гарантировать хотя бы одну итерацию.

4. Управление циклами: break и continue

Kotlin предоставляет ключевые слова для контроля выполнения циклов:

- break — немедленно завершает цикл.
- continue — пропускает текущую итерацию и переходит к следующей.

Пример:

```
for (i in 1..10) {
    if (i == 5) break // Прервать цикл, когда i равно 5
    if (i % 2 == 0) continue // Пропустить чётные числа
    println("Нечётное число: $i")
}
```

Этот код выведет только нечётные числа от 1 до 4, так как цикл прервётся на $i = 5$.

5. Вложенные циклы

Циклы можно вкладывать друг в друга для обработки сложных структур данных, таких как матрицы или иерархии.

Пример:

```
for (i in 1..3) {
    for (j in 1..3) {
        print("($i, $j) ")
    }
    println()
}
```

Этот код выведет координаты в виде матрицы:

(1, 1) (1, 2) (1, 3)
(2, 1) (2, 2) (2, 3)
(3, 1) (3, 2) (3, 3)

Менеджеры компоновки и Адаптеры в Android

1. Менеджеры компоновки (Layout Managers)

Менеджеры компоновки — это контейнеры, определяющие, как элементы интерфейса будут расположены на экране. Они наследуются от класса `ViewGroup` и реализуют различные стратегии позиционирования, обеспечивая гибкость и адаптивность макетов. Основные функции `Layout Managers` следующие:

- Управление расположением элементов (вертикальное, горизонтальное, относительное, сеточное и др.).
- Оптимизация отрисовки для повышения производительности.
- Поддержка динамических изменений макета в зависимости от контекста (например, смена ориентации экрана).

Рассмотрим принцип работы `Layout Managers`. Менеджеры компоновки вычисляют размеры и позиции дочерних элементов, учитывая их параметры (`layout_width`, `layout_height`) и правила расположения (например, относительное позиционирование или ограничения). Это позволяет создавать макеты, которые корректно отображаются на устройствах с разными экранами.

Ключевые особенности менеджеров компоновки:

- `ConstraintLayout` — современный инструмент для создания сложных макетов с минимальной вложенностью.
- `LinearLayout` — простой способ организации элементов в линию.
- `RecyclerView.LayoutManager` — специализированный менеджер для списков и сеток, поддерживающий прокрутку и переиспользование элементов.
- `FrameLayout` — базовый контейнер для наложения элементов.

Выбор менеджера компоновки зависит от задачи: для списков подходит `RecyclerView.LayoutManager`, для адаптивных макетов — `ConstraintLayout`, а для простых вертикальных/горизонтальных компоновок — `LinearLayout`.

2. Адаптеры (Adapters)

Адаптеры связывают данные приложения с элементами интерфейса, обеспечивая динамическое обновление UI при изменении данных. Они реализуют паттерн `ViewHolder`, который оптимизирует работу со списками, предотвращая повторные вызовы методов вроде `findViewById`. Адаптеры выполняют следующие основные задачи:

- Привязка данных к элементам макета.
- Управление жизненным циклом элементов (создание, обновление, удаление).
- Обработка изменений данных с минимальными накладными расходами.

Адаптер получает данные (например, список объектов) и предоставляет их для отображения в интерфейсе. При изменении данных адаптер обновляет только те элементы, которые действительно изменились, что снижает нагрузку на систему.

Ключевые компоненты адаптеров следующие:

- `ViewHolder`: Хранит ссылки на элементы макета, исключая лишние операции поиска.
- `RecycledViewPool`: Пул переиспользуемых элементов, уменьшающий частоту создания новых объектов.
- `DiffUtil`: Инструмент для вычисления разницы между старыми и новыми данными, минимизирующий обновления UI.

Адаптеры особенно эффективны в списках и сетках, где требуется отображать большие объёмы данных. Они обеспечивают плавную прокрутку и экономию ресурсов устройства.

3. Взаимодействие менеджеров и адаптеров

Менеджеры компоновки и адаптеры работают вместе, чтобы обеспечить эффективное отображение данных. Например, RecyclerView использует LayoutManager для определения расположения элементов и Adapter для привязки данных к каждому элементу:

- LayoutManager вычисляет, какие элементы видимы на экране и как они расположены.
- Adapter предоставляет данные для этих элементов, создавая или обновляя их представления.
- ViewHolder хранит ссылки на элементы, ускоряя доступ к ним.

Эта связь позволяет создавать динамические и производительные интерфейсы, где данные и визуализация отделены друг от друга.

Компонент RecyclerView

RecyclerView — это мощный компонент Android, предназначенный для отображения больших наборов данных с высокой производительностью. Он является частью библиотеки `androidx.recyclerview:recyclerview` и заменяет устаревший ListView, предлагая гибкость, оптимизацию и расширяемость. Понимание его структуры и принципов работы позволяет создавать эффективные и масштабируемые приложения, адаптированные под разные сценарии использования. RecyclerView построен на трёх ключевых элементах, которые взаимодействуют для отображения данных:

- 1) LayoutManager управляет расположением элементов на экране. Определяет, как элементы будут позиционироваться: вертикально, горизонтально, или в виде сетки:
 - LinearLayoutManager — вертикальный или горизонтальный список.
 - GridLayoutManager — сетка с заданным количеством столбцов.
 - StaggeredGridLayoutManager — стаггеренная сетка (например, для Pinterest-стиля).

- 2) Adapter связывает данные приложения с элементами интерфейса. Отвечает за создание и обновление ViewHolder, предоставляя данные для отображения. Имеются следующие ключевые методы:
- onCreateViewHolder() — создаёт новый ViewHolder.
 - onBindViewHolder() — привязывает данные к ViewHolder.
 - getItemCount() — возвращает количество элементов в списке.
- 3) ViewHolder хранит ссылки на элементы интерфейса (например, TextView, ImageView) внутри одного элемента списка. Предотвращает повторные вызовы findViewById(), что ускоряет работу.

RecyclerView оптимизирует отрисовку списка, используя следующие механизмы:

- Переиспользование ViewHolder:

Когда элемент уходит за пределы экрана, он добавляется в пул (RecycledViewPool) и используется для новых элементов, появляющихся при прокрутке. Это уменьшает частоту создания новых объектов и снижает нагрузку на память.

- Лазерная прорисовка:

RecyclerView обновляет только те элементы, которые действительно изменились, а не весь список целиком. Это достигается через методы вроде notifyItemInserted() или notifyItemChanged() вместо notifyDataSetChanged(), что позволяет анимировать изменения.

- Разделение обязанностей:

Логика расположения (LayoutManager), данных (Adapter) и представления (ViewHolder) разделена, что упрощает расширяемость и тестирование.

Основные Преимущества RecyclerView заключаются в следующем:

- Высокая Производительность

За счёт переиспользования ViewHolder и лазерных обновлений RecyclerView работает быстрее, чем ListView.

- Гибкость Расположения

Поддержка различных `LayoutManager` позволяет легко переключаться между вертикальным списком, горизонтальной прокруткой, сеткой и кастомными макетами.

- **Поддержка Анимаций**

`RecyclerView` автоматически анимирует добавление, удаление и изменение элементов, если использовать методы вроде `notifyItemInserted()`.

- **Расширяемость**

Возможность кастомизации через `ItemDecorator` (для разделителей, отступов) и `ItemAnimator` (для анимаций).

- **Интеграция с Современными Паттернами**

Использование `DiffUtil` для вычисления разницы между старыми и новыми данными, что минимизирует обновления UI.

Коллекции в Kotlin

Коллекции в Kotlin — это структуры данных, предназначенные для хранения и управления наборами элементов. Они реализованы в стандартной библиотеке Kotlin и предоставляют широкий набор возможностей для работы с данными. Коллекции делятся на три основных типа: списки (`List`), множества (`Set`) и карты (`Map`). Каждая коллекция имеет неизменяемую (`read-only`) и изменяемую (`mutable`) версию, что позволяет контролировать модификацию данных в зависимости от контекста использования. Рассмотрим типы коллекций и их возможности.

Списки (List)

Списки представляют собой упорядоченные коллекции, где элементы доступны по индексу. Индексация начинается с 0, а дублирование элементов разрешено.

Неизменяемый список создаётся с помощью функции `listOf()`. После инициализации его содержимое нельзя модифицировать: попытки добавления, удаления или изменения элементов приведут к ошибке компиляции. Это особенно полезно для защиты данных, которые не должны

изменяться, например, конфигурационные параметры или статические справочники.

Изменяемый список реализуется через `mutableListOf()`. Он предоставляет методы для динамического управления элементами:

`add(element)` — добавляет элемент в конец списка или по указанному индексу.

`remove(element)` — удаляет элемент.

`set(index, element)` — заменяет элемент по индексу.

`sort()` — сортирует элементы.

Такой подход позволяет гибко управлять данными, например, добавлять новые элементы в реальном времени или обновлять их значения.

Кроме того, списки поддерживают индексный доступ через оператор `[]`, что упрощает получение и изменение элементов.

Множества (Set)

Множества хранят уникальные элементы без повторений. Порядок элементов в множестве не гарантируется. Неизменяемое множество создаётся через `setOf()`. После инициализации его содержимое остаётся неизменным. Это полезно для хранения фиксированных наборов данных, таких как перечни разрешённых операций или список запрещённых слов.

Изменяемое множество реализуется с помощью `mutableSetOf()`. Оно позволяет:

- Добавлять элементы с помощью `add()`.
- Удалять элементы через `remove()`.
- Проверять наличие элемента с `contains()`.

Особенностью множеств является то, что уникальность элементов определяется через методы `equals()` и `hashCode()`. При использовании пользовательских классов важно правильно реализовать эти методы, чтобы избежать неожиданных результатов. Кроме того, множества могут быть реализованы через `HashSet` (не гарантирует порядок) или `LinkedHashSet` (сохраняет порядок вставки). Выбор между ними зависит от контекста:

HashSet — если порядок не важен, а нужна максимальная скорость операций.

LinkedHashSet — если требуется сохранить последовательность добавления.

Карты (Map)

Карты представляют собой набор пар «ключ-значение», где ключи уникальны, а доступ к значениям осуществляется через ключи. Неизменяемая карта создаётся через `mapOf()`. После инициализации её содержимое нельзя изменить, что полезно для статических данных, таких как константы или конфигурации.

Изменяемая карта реализуется с помощью `mutableMapOf()`. Она поддерживает:

- Добавление пар через `put(key, value)`.
- Удаление пар с помощью `remove(key)`.
- Получение значения по ключу через `get(key)` или `[]`.

Карты также предоставляют методы для работы с ключами и значениями:

`keys` — получение множества ключей.

`values` — извлечение значений.

`entries` — доступ ко всем парам.

Важно учитывать, что ключи в картах сравниваются через `equals()` и `hashCode()`, как и в множествах. Это особенно актуально при использовании собственных классов в качестве ключей. Без корректной реализации методов сравнения карта не сможет найти значение, даже если ключи содержат одинаковые данные.

Карты также могут быть реализованы через `HashMap` (порядок не гарантирован) или `LinkedHashMap` (сохраняет порядок вставки). Эти реализации влияют на поведение карты, особенно при итерации:

```
val map1 = linkedMapOf("A" to 1, "B" to 2)
val map2 = hashMapOf("A" to 1, "B" to 2)
println(map1.keys) // Вывод: [A, B]
println(map2.keys) // Порядок может отличаться
```

Итак, выбор коллекции зависит от задач:

- List — если порядок элементов важен. Например, для отображения истории действий или хранения данных с последовательностью.
- Set — когда требуется уникальность элементов, например, для проверки дубликатов при регистрации пользователей.
- Map — если данные логически связаны с ключами, как в случае хранения информации о сотрудниках, где ключ — идентификатор, а значение — данные о человеке.

Изменяемые и неизменяемые коллекции

Kotlin строго разделяет read-only и mutable коллекции, что помогает избежать случайных изменений данных. Неизменяемые коллекции (listOf, setOf, mapOf) предоставляют доступ только для чтения. Любые попытки изменить их вызовут ошибку компиляции. Изменяемые коллекции (mutableListOf, mutableSetOf, mutableMapOf) позволяют добавлять, удалять и обновлять элементы.

Этот подход особенно важен при работе с данными, которые должны оставаться неизменными, например, в архитектуре MVVM или при передаче данных между слоями приложения.

Функции для обработки данных в коллекциях

filter — фильтрует элементы по условию.

map — преобразует элементы в новую форму.

sortedBy — сортирует элементы по заданному критерию.

groupBy — группирует элементы по ключу.

sumOf — вычисляет сумму значений.

Пример 1: Обработка данных о пользователях

```
// Создание списка пользователей с именами и возрастами
val users = listOf(
    mapOf("name" to "Alice", "age" to 25),
    mapOf("name" to "Bob", "age" to 30),
    mapOf("name" to "Charlie", "age" to 25),
    mapOf("name" to "David", "age" to 30)
```

```
)
// Фильтрация пользователей старше 25 лет
val filteredUsers = users.filter { it["age"] as Int > 25 }

// Группировка пользователей по возрасту
val groupedByAge = users.groupBy { it["age"] as Int }

// Вывод результатов
println("Фильтрованные пользователи: $filteredUsers")
println("Группировка по возрасту: $groupedByAge")
```

- В примере используется список карт (listOf(mapOf(...))) для хранения данных о пользователях.
- Метод filter() выбирает пользователей старше 25 лет, используя ключ "age".
- Метод groupBy() группирует пользователей по возрасту, создавая структуру, где ключ — возраст, а значение — список соответствующих пользователей.
- Результаты выводятся в консоль, демонстрируя эффективность работы с коллекциями.

Пример 2: Анализ статистики продаж

```
// Создание изменяемой карты для хранения продаж по месяцам
val sales = mutableMapOf(
    "Январь" to listOf(120, 150, 200),
    "Февраль" to listOf(180, 210, 130),
    "Март" to listOf(220, 250, 300)
)

// Добавление новых данных о продажах
sales["Апрель"] = listOf(200, 230, 270)

// Вычисление общей суммы продаж за каждый месяц
val totalSales = sales.mapValues { (_, months) -> months.sum() }

// Нахождение месяца с максимальной суммой продаж
val maxMonth = totalSales.maxByOrNull { it.value }?.key ?: "Нет данных"

// Удаление данных за январь
sales.remove("Январь")
```

```
// Вывод результатов
println("Общие продажи: $totalSales")
println("Месяц с максимальными продажами: $maxMonth")
println("Обновлённые данные: $sales")
```

- В примере используется `mutableMapOf()` для создания карты, где ключ — месяц, значение — список продаж.
- Метод `mapValues()` вычисляет общую сумму продаж для каждого месяца, применяя `sum()` к спискам значений.
- `maxByOrNull()` находит месяц с максимальной суммой, обрабатывая случай отсутствия данных через `?..`
- Метод `remove()` удаляет данные за январь, демонстрируя изменяемость карты.
- Вывод результатов показывает, как коллекции упрощают анализ и обработку данных.

Одним из главных преимуществ коллекций в Kotlin является безопасность типов, которая гарантирует, что в коллекции могут храниться только элементы заранее заданного типа. Это исключает возможность случайного добавления данных несовместимого типа, что особенно важно при работе с большими объёмами информации.

Например, если объявлен список `List<String>`, попытка добавить в него число или объект другого класса вызовет ошибку на этапе компиляции. Такая строгость не только предотвращает логические ошибки, но и упрощает понимание кода, так как разработчик всегда точно знает, с какими данными работает. Ещё одной важной стороной коллекций является их высокая производительность, достигаемая за счёт использования встроенных методов, таких как `filter`, `map`, `sortedBy` и других. Эти функции оптимизированы для работы с данными и позволяют избежать ручного написания циклов, что снижает количество кода и повышает его читаемость. Например, вместо того чтобы писать вложенные циклы для фильтрации или преобразования элементов, достаточно вызвать соответствующий метод, который реализует логику за вас. Более того, такие методы часто используют

эффективные алгоритмы, что положительно сказывается на скорости выполнения задач.

Выбор конкретного типа коллекции играет решающую роль в эффективности и корректности решения. Списки (List) применяются, когда порядок элементов имеет значение, например, для отображения исторических данных, где последовательность событий критична. Множества (Set) становятся идеальным выбором, если требуется обеспечить уникальность элементов, как в случае проверки дубликатов при регистрации пользователей или обработке набора идентификаторов. Карты (Map) востребованы тогда, когда данные логически связаны с ключами, например, для хранения информации о сотрудниках, где ключом может быть идентификатор, а значением — их данные. Правильный выбор коллекции позволяет не только избежать избыточной обработки, но и упрощает доступ к данным, так как каждая структура оптимизирована под свой сценарий.

Интеграция коллекций с функциональными методами делает их ещё более мощным инструментом. Такие методы как `groupBy`, `sumOf`, `firstOrNull` и `flatMap` позволяют решать сложные задачи с минимальными усилиями, сохраняя чистоту и выразительность кода. Например, `groupBy` может разделить элементы по категориям, `sumOf` — подсчитать общее значение, а `firstOrNull` — найти первый подходящий элемент, избегая лишних проверок. Эти методы не только упрощают логику, но и минимизируют вероятность ошибок, связанных с ручной реализацией таких операций.

Программное создание кастомизированного интерфейса с помощью Kotlin в Android Studio

Традиционно интерфейс Android-приложений создаётся с использованием XML-разметки, где элементы интерфейса (кнопки, текстовые поля, контейнеры) описываются в декларативном виде. Однако в некоторых случаях возникает необходимость динамического создания и настройки элементов в коде Kotlin, особенно при реализации сложных сценариев:

- Добавление элементов в реальном времени (например, чат с динамически появляющимися сообщениями).
- Адаптация макета под разные устройства или ориентации экрана.
- Создание уникальных анимаций или переходов.
- Реализация кастомных компонентов, не поддерживаемых стандартными библиотеками.

Программное создание интерфейса требует понимания работы с View, ViewGroup, LayoutParams и методами жизненного цикла активности. Этот подход даёт максимальную гибкость, но требует аккуратности для избежания утечек памяти и снижения производительности. View — это базовый класс для всех элементов интерфейса: кнопок, текстовых полей, изображений. ViewGroup — абстрактный класс, являющийся родительским для всех контейнеров (например, LinearLayout, ConstraintLayout). Он управляет расположением и отрисовкой дочерних элементов. Для программного управления расположением элементов в контейнере используются LayoutParams — классы, определяющие размеры и позиционирование.

Интерфейс создаётся в методе onCreate() активности, где через setContentView() устанавливается корневой элемент. При программном подходе можно обойтись без XML, создавая макет динамически. Рассмотрим данный процесс по шагам:

Шаг 1: Инициализация контейнера

Корневой контейнер (например, LinearLayout) создаётся в коде и устанавливается как корневой элемент через setContentView().

Шаг 2: Создание дочерних элементов

Элементы интерфейса (кнопки, текстовые поля, изображения) создаются программно и настраиваются через свойства: текст, цвет, размеры, отступы.

Шаг 3: Настройка параметров макета

Для каждого элемента указываются параметры расположения, такие как ширина, высота, отступы, вес (weight) для LinearLayout.

Шаг 4: Добавление элементов в контейнер

Элементы добавляются через метод `addView()` контейнера.

Шаг 5: Обработка событий

События, такие как клики, прокрутка или анимации, реализуются через лямбда-выражения или слушатели событий.

Пример 1: Динамический список кнопок. Этот пример демонстрирует создание вертикального списка кнопок, добавляемых в `LinearLayout` программно.

```
class MainActivity : AppCompatActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)

        // Создание вертикального LinearLayout как корневого контейнера
        val container = LinearLayout(this).apply {
            orientation = LinearLayout.VERTICAL
            layoutParams = LinearLayout.LayoutParams(
                LinearLayout.LayoutParams.MATCH_PARENT,
                LinearLayout.LayoutParams.WRAP_CONTENT
            ).apply {
                setPadding(16, 16, 16, 16)
            }
        }

        // Добавление кнопок в контейнер
        for (i in 1..5) {
            val button = Button(this).apply {
                text = "Кнопка $i"
                textSize = 18f
                setBackgroundColor(ContextCompat.getColor(this@MainActivity,
                    android.R.color.holo_blue_dark))
                setTextColor(ContextCompat.getColor(this@MainActivity,
                    android.R.color.white))

                // Обработка клика
                setOnClickListener {
                    Toast.makeText(this@MainActivity, "Нажата кнопка $i",
                        Toast.LENGTH_SHORT).show()
                }
            }

            // Настройка параметров макета для кнопки
            val params = LinearLayout.LayoutParams(
                LinearLayout.LayoutParams.MATCH_PARENT,
                LinearLayout.LayoutParams.WRAP_CONTENT
```

```

        ).apply {
            bottomMargin = 16 // Отступ между кнопками
        }
        // Добавление кнопки в контейнер
        container.addView(button, params)
    }
    // Установка контейнера как корневого элемента
    setContentView(container)
}
}

```

- LinearLayout используется как корневой контейнер с вертикальной ориентацией.
- Button создаётся в цикле с настройкой текста, цвета и обработчика клика.
- LayoutParams определяют размеры и отступы кнопок.
- setContentView() устанавливает контейнер как основной макет активности.
- Toast выводит сообщение при нажатии на кнопку.

Этот подход позволяет динамически генерировать элементы, например, загружать данные с сервера и отображать их в виде кнопок.

Пример 2: Кастомная анимированная карточка. Этот пример иллюстрирует создание кастомной карточки с изображением, текстом и анимацией появления.

```

class MainActivity : AppCompatActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)

        // Создание ConstraintLayout как корневого контейнера
        val container = ConstraintLayout(this).apply {
            layoutParams = ConstraintLayout.LayoutParams(
                ConstraintLayout.LayoutParams.MATCH_PARENT,
                ConstraintLayout.LayoutParams.MATCH_PARENT
            )
        }

        // Создание ImageView с изображением
        val imageView = ImageView(this).apply {
            setImageResource(R.drawable.ic_launcher_foreground) // Загрузка из
ресурсов
            adjustViewBounds = true
            scaleType = ImageView.ScaleType.CENTER_CROP

```



```

        id = View.generateViewId() // Генерация уникального ID
    }
    // Создание TextView с заголовком
    val textView = TextView(this).apply {
        text = "Кастомная карточка"
        textSize = 20f
        setTextColor(ContextCompat.getColor(this@MainActivity,
android.R.color.black))
        id = View.generateViewId()
    }
    // Настройка параметров для ImageView в ConstraintLayout
    val imageParams = ConstraintLayout.LayoutParams(
        150,
        150
    ).apply {
        topToTop = ConstraintLayout.LayoutParams.PARENT_ID
        startToStart = ConstraintLayout.LayoutParams.PARENT_ID
        endToEnd = ConstraintLayout.LayoutParams.PARENT_ID
        topMargin = 50
    }
    // Настройка параметров для TextView в ConstraintLayout
    val textParams = ConstraintLayout.LayoutParams(
        ConstraintLayout.LayoutParams.WRAP_CONTENT,
        ConstraintLayout.LayoutParams.WRAP_CONTENT
    ).apply {
        topToBottom = imageView.id
        startToStart = ConstraintLayout.LayoutParams.PARENT_ID
        endToEnd = ConstraintLayout.LayoutParams.PARENT_ID
        topMargin = 16
    }
    // Добавление элементов в контейнер
    container.addView(imageView, imageParams)
    container.addView(textView, textParams)

    // Анимация появления карточки
    val animator = ObjectAnimator.ofFloat(imageView, "alpha", 0f, 1f).apply {
        duration = 500
        interpolator = DecelerateInterpolator()
    }
    // Запуск анимации после отрисовки
    imageView.post { animator.start() }

    // Установка контейнера как корневого элемента
    setContentView(container)
}

```

}

- `ConstraintLayout` выбран для точного позиционирования элементов.
- `ImageView` и `TextView` создаются программно и позиционируются относительно друг друга через `ConstraintLayout.LayoutParams`.
- `generateViewId()` обеспечивает уникальные ID для элементов.
- `ObjectAnimator` реализует анимацию прозрачности изображения.
- `post()` гарантирует запуск анимации после отрисовки элемента.

Этот пример демонстрирует, как сочетать программное создание интерфейса с анимациями и кастомными стилями, недоступными через XML.